

Practical `tick()` Scenario: Auto-Scrolling a Chat Log

Q48

SVELTE 5 TICK DOM SCROLL PRACTICAL

INTERVIEW QUESTION

Question. Provide a practical scenario where awaiting `tick()` is required to read accurate DOM states after a state change.

ANSWER.

The scenario — a chat log that must auto-scroll to the bottom

- **The feature:** a chat component renders an array of messages with `{#each}`, and every time a new message arrives, the container must scroll to the **bottom** so the latest message is visible.
- **The measurement:** to scroll to the bottom you read `** el.scrollHeight (the full height of the content) and assign it to el.scrollTop (the scroll position). This is a DOM read followed by a DOM write**` that depends on the read.
- **The trap:** calling `messages.push(newMsg)` dirties state, but the new `<li>` is **not in the DOM yet** because the flush is scheduled for after the handler returns (see Q46). Reading `scrollHeight` at that instant returns the **pre-push height** — the new message is invisible, and the scroll lands one message short.

****The fix — `await tick()` between the write and the read****

- **Yield before measuring:** insert `** await tick() **` between the state change and the DOM read. `tick()` returns a **Promise** that resolves once the current **reactive flush** finishes — the flush being Svelte's term for the moment the new `<li>` is actually written to the DOM.

- **Now the read is accurate:** by the time execution resumes, the DOM reflects the new array, so `el.scrollHeight` includes the freshly pushed message, and `el.scrollTop = el.scrollHeight` lands exactly at the bottom.
- **The handler must be `async` and `await`** only works inside a function declared `async`, so the click/send handler must be `async function send()` rather than a plain function.

Why you cannot fake it with `setTimeout`

- `setTimeout(fn, 0)` is wrong: it schedules the callback on the macrotask queue, which runs after rendering — meaning the user sees one frame of the old scroll position before it jumps. `tick()` resolves on the microtask queue, before paint, so the scroll updates in the same frame as the new message.
- **No debounce either:** wrapping the scroll in a timeout adds visible latency that feels broken. `tick()` adds none — it waits only for the flush that was already going to happen.

Chat.svelte

```
<script>
  import { tick } from 'svelte';

  let messages = $state(['hello', 'world']);
  let logEl;

  async function send() {
    messages.push('new message'); // State changed; <li> not
    in DOM yet.
    await tick(); // Wait for the flush to
    write the new <li>.
    logEl.scrollTop = logEl.scrollHeight; // Now scrollHeight
    includes the new row.
  }
</script>

<ul bind:this={logEl} style="height:120px; overflow:auto;">
  {#each messages as m}<li>{m}</li>{/each}
</ul>
```

```
<button onclick={send}>send</button>
```

① **Summary.** Auto-scrolling a chat log after `messages.push(...)` requires reading `scrollHeight`, but the new `<li>` is not in the DOM until the flush runs. `await tick()` between the push and the read guarantees the measurement reflects the new state — and unlike `setTimeout`, it resolves before paint, so there is no visible latency.

